Ders 13 Genişletilmiş Notu: C Programlama Diline Giriş: Yapı, Operatörler, Tipler, if ve Döngüler

Ders 13 Genişletilmiş Notu: C Programlama Diline Giriş: Yapı, Operatörler, Tipler, if ve Döngüler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Programlama Dillerinin Sınıflandırılması
- 2. Derlenen ve Yorumlanan Diller
- 3. C Dilinin Kısa Tarihi ve Gücü
- 4. Bir C Programının Temel Yapısı
- 5. Operatörler
- 6. Değişken Tipleri
- 7. Format Belirteçleri
- 8. Geliştirme Ortamları
- 9. Koşullu Dallanma: if ve else
- 10. İç İçe if: Üç Sayının En Küçüğü
- 11. Döngüler: while ve do…while
- 12. Döngüler: for
- 13. C'de Dizi İndisleri
- 14. Derleyici ile IDE Farkı Üzerine Son Soru

1. Programlama Dillerinin Sınıflandırılması

Hoca, uygulama saatinde soru çözmek yerine C programlama diline giriş yapacaklarını ve gerek duyulacak bilgileri hızlıca aktaracağını söyleyerek başladı. Bu dönem C dilinin yalnızca en temel özellikleri görülecek. Amaç, akış diyagramlarını ve algoritmaları canlandırmak, onları bilgisayara çözdürmektir. Bu yüzden diller hakkındaki genel bilgiler de fazla ayrıntıya girmeyecek.

1.1 Biçimsel Diller

Programlama dilleri biçimsel dillerdir (formal languages). Biçimsel bir dilde kullandığınız ifade her zaman tek bir anlama gelir. Türkçede, İngilizcede ya da başka bir doğal dilde bir kelime birden fazla anlama gelebilir. Programlama dilinde ise böyle bir ifadeyle karşılaşmazsınız. Bilgisayar ifadeyi ne ise o şekilde değerlendirir ve karşılığında yapılması gereken işleri yapar. Sözdizimi (syntax) açısından tek bir ifade tek bir şekilde çalıştırılır.

1.2 Dil Aileleri

Hoca iki temel aile saydı: yapısal programlama dilleri (structured programming languages) ve nesneye yönelik programlama dilleri (object-oriented programming languages). Bunların ötesinde başka gruplar da vardır. Örneğin gömülü sistemler gibi belirli ortamlar için yazılmış programlama dilleri de bulunur. Hoca, biçimsel diller hakkında araştırma yapılabileceğini de ekledi.

Hoca toparlarken üçüncü bir kategori olarak doğal dilleri de saydı ve hepsinin biçimsel dil olduğunu söyledi.

> **Ders sonrası not:** Hoca doğal dilleri programlama dillerinin üçüncü kategorisi olarak saydı ve üçünü de biçimsel dil diye adlandırdı. Doğrusu büyük olasılıkla şudur: Doğal diller programlama dili değildir, biçimsel dillerin karşıtıdır. Hoca da aynı anlatımda doğal dillerde bir ifadenin birden fazla anlama gelebildiğini söyleyerek bu ayrımı yapıyor.

1.3 Alt, Orta ve Yüksek Seviye

Diller donanıma yakınlıklarına göre alt seviye, orta seviye ve yüksek seviye olarak ayrılır. Bilgisayar donanımından ve işlemci üzerinde çalışan komut setinden (CPU instruction set) uzaklaştıkça yüksek seviyeli, konuşma diline yakın dillere yaklaşırsınız. Bu ayrımın iki yönü vardır:

- **Alt seviye diller:** İlk bakışta anlaşılması zordur ve uzun kod yazmayı gerektirir. Buna karşılık sistemi en verimli şekilde kullanmanızı sağlar.
- **Yüksek seviye diller:** Oldukça rahat anlaşılır. Bir işi birkaç kısa komutla yapmanızı sağlar.

Her seviyeden örnekler şöyledir:

- **Alt seviye:** En altta makine kodu vardır, yani 0 ve 1'lerden oluşan komut setleri. Bunların kombinasyonlarını yazmak ve yönetmek çok zordur. Alt seviyenin en bilinen dillerinden biri assembly dilidir.
- **Orta seviye:** C. Bazı kaynaklarda yüksek seviye olarak da geçer.
- **Yüksek seviye:** Java, SQL ve benzeri birçok dil.

C genelde orta seviye bir dil olarak geçer. Çünkü bir yandan tüm sistem kaynaklarını yönetmenizi sağlar, öte yandan alt seviye dillere göre görece daha anlaşılırdır ve daha az kod yazmayı gerektirir. Python ile karşılaştırıldığında ise durum değişir. Python C dili üzerine inşa edilmiştir ve çok daha az kod yazarak çok daha fazla iş yapabilir.

2. Derlenen ve Yorumlanan Diller

Diller kendi içinde ikiye ayrılır: derleyici (compiler) kullanan derlenen diller (compiled languages) ve yorumlayıcı (interpreter) kullanan yorumlanan diller (interpreted languages). Hoca farkı önce öğrencilere sordu.

Soru: Derlenen ve yorumlanan diller arasındaki fark nedir? Bir öğrencinin cevabı: Derlenen diller yazılan kodu alıp doğrudan makine diline çevirir, yorumlanan diller ise kodu başka bir dilin koduna ya da bytecode denen bir koda çevirip öyle çalıştırır. **Cevap:** Hoca ilk kısmın doğru olduğunu, sonraki kısmın ise biraz fazladan olduğunu söyledi ve iki yapıyı tahtada anlattı.

2.1 Derlenen Diller

Bir kaynak kod (source file) yazarsınız. Programlama dilinin sözdizimini kullanarak yazdığınız bu dosyaya kaynak kod denir. Kullandığınız IDE ya da derleyici komutları üzerinden bu kodu derleyiciye verirsiniz. Akış şöyledir:

1. Derleyici kaynak kodu nesne koduna (object code) dönüştürür. Birden fazla kaynak dosya varsa birden fazla nesne kodu oluşur.
2. Bağlayıcı (linker) bu nesne kodlarını birbiriyle ilişkilendirip tek bir çalıştırılabilir koda (executable) çevirir. Hangisinin önce, hangisinin sonra çalışacağını siz kodu yazarken zaten belirtmiş olursunuz.

Sonuçta kaç nesne kodu olursa olsun en sonunda mutlaka tek bir çalıştırılabilir kod elde edilir. Tek kaynak dosya varsa bir nesne kodu ve bir çalıştırılabilir kod üretilir. Bu işlem otomatik olarak gerçekleşir. C programlama dili bu mantığı kullanır.

Derlenen diller, yorumlanan dillere göre daha verimli ve daha hızlı çalışan kodlar üretir.

2.2 Yorumlanan Diller

Yorumlanan dillerde komutlar satır satır, adım adım dönüştürülerek o anda çalıştırılır. Yorumlayıcı kaynak kodu alır, içindeki ilk komutu dönüştürür ve bilgisayarda çalıştırır (execute), sonra sıradaki komuta geçer.

Bunun dezavantajı performanstır. Kodun tamamı tek seferde makine diline çevrilmediği, çevirme işi adım adım yapıldığı için yorumlanan diller derlenen dillere göre daha yavaş çalışır. Hocanın verdiği en iyi örneklerden biri MATLAB'dir.

2.3 Hem Derlenen Hem Yorumlanan Diller: Java

İkisini birden kullanan diller de vardır. Hoca buna örnek istedi.

Soru: Hem derlenen hem yorumlanan bir dile ne örnek verilebilir? Öğrencilerden gelen cevaplar sırayla Python, JavaScript ve Java oldu. **Cevap:** Python ve JavaScript örnek sayılmaz, ikisi de yorumlanan dildir. Doğru cevap Java'dır. Bunu söyleyen öğrenci, Java'nın kendi içinde bir sanal makine açıp kodları orada çalıştırdığını, buna JVM dendiğini hatırlattı.

> **Ek bilgi (derste anlatılmadı):** Python'un yaygın uygulaması olan CPython da büyük olasılıkla kaynak kodu önce bytecode'a çevirir, sonra bu bytecode'u sanal makinesinde yorumlar. Modern JavaScript motorları da büyük olasılıkla kodu çalışma anında derler. Yani "yorumlanan" etiketi dilin kendisinden çok, tipik uygulamasını anlatan bir basitleştirmedir. Giriş düzeyinde hocanın sınıflandırması yeterlidir.

Java Virtual Machine (JVM), Java dili için oluşturulmuş bir yapıdır. Hoca bunu aslında sanal bir işlemci olarak tarif etti. Amacı, "bir bilgisayarda yaz, geri kalan bilgisayarlarda kullan" mantığıyla çalışmaktır: Kodu Windows'ta yazıp Linux'ta çalıştırabilirsiniz.

Bunun için iki aşama gerekir:

1. Önce derleme (compilation) yapılır: Kod, JVM'in anlayacağı bytecode'a dönüştürülür.
2. Sonra yorumlama (interpret) yapılır: Dönüştürülmüş komutlar tek tek yorumlanarak makine koduna çevrilir.

   Hocanın vurgusu: Java platformdan bağımsızdır (platform independent). Bu önemlidir.

C böyle bir dil değildir. Belirli bir işlemci mimarisi için derlediğiniz kodun ürettiği çalıştırılabilir dosya yalnızca o mimaride çalışır. Başka bir mimaride çalıştırmak isterseniz tekrar derlemeniz gerekir. Java'da ise bir kodu istediğiniz ortama aktarıp ek bir işlem yapmadan çalıştırabilirsiniz.

3. C Dilinin Kısa Tarihi ve Gücü

C programlama dili 1972 yılında Unix işletim sistemini yazmak için ortaya atılmıştır. İlk yazılan hali B dili diye geçer, bir sonraki sürüm C olarak adlandırılmıştır.

Hoca, bir dille işletim sistemi yazabiliyorsanız bütün kaynaklara erişebiliyorsunuz demektir, dedi. C gücünü buradan alır: Hem bellek yönetiminde (memory management) hem de kaynaklara doğrudan ulaşımda önemli artılar getirir.

Yaklaşık 50 senedir hem popülerliğini ve kullanım miktarını korumuş hem de başka dillere ilham vermiştir. Java, C#, C++ ve başka birçok dilde benzer sözdizimleri kullanılır. Python gibi diller ise C üzerine inşa edilmiştir.

C hem çok güçlüdür hem de kendine has birçok özelliği vardır. Bu özelliklere yeterince hâkim olmadığınızda yazdığınız programların yanlış çalışma olasılığı vardır.

4. Bir C Programının Temel Yapısı

4.1 Program İskeleti

Hoca bir kaynak dosyada mutlaka bulunması gerekenleri sırayla gösterdi.

Kütüphaneler. Kütüphaneler `#` işareti ve `include` ile eklenir. `stdio.h` olmazsa olmazdır, çünkü `printf` ve `scanf` gibi ekrana yazdırma ve kullanıcıdan okuma fonksiyonlarını içerir. Başka kütüphaneler de vardır, örneğin `stdlib.h` ve `string.h`.

> **Ders sonrası not:** Hoca `stdio.h`'nin her programa mutlaka eklenmesi gerektiğini söyledi. Doğrusu büyük olasılıkla şudur: Bu kütüphane giriş ve çıkış fonksiyonlarını kullanan programlar için gereklidir. Bu dersteki programların hepsi bu fonksiyonları kullandığı için hocanın önerisi pratikte geçerlidir.

Ana fonksiyon. Her C kodunda `main` adında bir ana fonksiyon mutlaka bulunur. Fonksiyonlar süslü parantezle açılıp kapanır.

Dönüş değeri. `int main()` yazımındaki `int`, ana fonksiyonun bir tam sayı değer döndüreceğini söyler. `return 0;` ile 0 değerini döndürdüğünüzde programın sorunsuz bittiğini ve kontrolün işletim sistemine devredilebileceğini bildirmiş olursunuz. Yani fonksiyonun bir adı (`main`), döndürdüğü değere göre bir tipi (`int`) vardır ve değer döndürmek için `return` anahtar kelimesi kullanılır.

4.2 İki Sayıyı Toplayan Program

Hoca bu iskelet üzerine basit bir toplama programı yazdı. Akış diyagramlarından farklı olarak C'de değişken mutlaka tanımlanmalıdır. Bazı dillerde değişkeni tanımlamadan, adını yazdığınız yerde kullanabilirsiniz. C'de bu mümkün değildir.

```c
#include <stdio.h>
int main() {
    int sayi1, sayi2, sonuc;
    printf("Bir deger giriniz"); // Comment Line: Ilk sayiyi giriniz
    scanf("%d", &sayi1);
    // Comment Line: 2. sayiyi giriniz
    printf("Ikinci degeri giriniz");
    scanf("%d", &sayi2);
    sonuc = sayi1 + sayi2;
    printf("Sonuc = %d", sonuc);
    return 0;
}
```

Hoca programı yazarken önce yalnızca `sayi1` ve `sayi2`'yi tanımlamıştı ve neyin eksik olduğunu sordu.

Soru: Bu programda eksik olan nedir? **Cevap:** Bir öğrenci `sonuc` değişkeninin tanımlanmadığını söyledi. Hoca tanımı ekledi ve böylece toplama yapan basit bir C programı yazılmış oldu.

4.3 scanf, printf ve Adres İşareti

`scanf` ile değeri dışarıdan okursunuz. Okuma için çeşitli formatlar vardır, burada `%d` kullanıldı. `&sayi1` içindeki & (ampersand) işareti, okumanın `sayi1`'in adresine yapılacağını söyler.

> **Hocanın vurgusu:** `scanf` ile bir değişkene okuma yaparken bu işareti mutlaka kullanacağız. Tek bir istisna var, onu yeri geldiğinde öğreneceksiniz. `printf`'lerde ise adres işareti kullanmıyoruz.
>
> **Ek bilgi (derste anlatılmadı):** Hocanın ileriye bıraktığı istisna büyük olasılıkla karakter dizileridir. `char kelime[10];` için `scanf("%s", kelime);` yazılır, çünkü dizinin adı zaten bir adres belirtir.

`printf("Sonuc = %d", sonuc);` satırında `sonuc` değişkeni `%d` ile ilişkilidir. `printf` önce `Sonuc =` yazar, `%d`'yi gördüğü yere de `sonuc` değişkeninin değerini yazar.

4.4 Açıklama Satırları

`//` işaretinden sonrası açıklama (comment) olur. Açıklama, kodun yanına ya da ayrı bir satıra yazılabilir. Blok halinde açıklama için `/*` ile `*/` kullanılır, bu ikisi arasına yazdığınız bütün satırlar açıklama sayılır.

Açıklamalar kaynak dosyada durur ama derleme işlemine girmez. İki amacı vardır: Programı okuyan başka birinin anlaşılması güç noktaları daha rahat anlaması ve belli bir süre sonra programa döndüğünüzde ne yaptığınızı hatırlamanız.

> **Hocanın vurgusu:** Program yazarken unutmamanız gerekenler: `stdio.h` kütüphanesini eklemek, `main` fonksiyonunu açıp kapamak, `return 0` ile sonlandırmak, okuma ve yazma için `scanf` ve `printf` kullanmak. Bunların ötesinde de temel işlemleri yapmak.

5. Operatörler

Hoca şu an ihtiyaç duyulacak temel operatörleri ve gösterimleri dört grupta verdi.

- **Aritmetik işlemler (arithmetic operations):** `+`, `-`, `*`, `/` ve mod alma için `%`.
- **Atama (assignment):** `=`. Atama sağdan sola yapılır.
- **İlişkisel işlemler (relational operations):** `<`, `>`, `<=`, `>=`, eşit mi sorusu için `==`, eşit değil mi sorusu için `!=`. Bunlar sık kullanılacak.
- **Mantıksal işlemler (logic operations):** AND için iki ampersand `&&`, OR için iki dik çizgi `||`. Genelde `if` ve `while` koşullarında kullanılır.

6. Değişken Tipleri

Hoca tipleri hızlıca yazdı ve üzerinden daha sonra daha ayrıntılı geçeceklerini söyledi.

- **Tam sayılar:** `int`. Daha az yer kaplaması için `short int`, daha büyük sayılar için `long int`.
- **Kesirli sayılar:** `float`. Hassasiyet daha yüksek olsun isterseniz `double`, bunun bir ötesi `long double`.
- **Karakter:** `char`.

Birden fazla karakter için ayrı bir tip yoktur. Bunun için karakter dizisi oluşturmanız gerekir. Örneğin `char kelime[10];` yazarsanız birden fazla karakter saklayabilirsiniz.

6.1 unsigned Anahtar Kelimesi

Bu tiplere ek olarak `unsigned` anahtar kelimesi vardır: `unsigned int`, `unsigned char`, `unsigned long int` gibi. Bu tanımlamalar işaretsiz sayı saklamak içindir. Hoca bunların şu an kritik olmadığını ama karşılaşılacağını söyledi.

Karakter tipi C'de her zaman 1 byte yer kaplar. Hoca buradan bir soru sordu.

Soru: `unsigned char` dediğimizde hangi sayı aralığını saklayabilirsiniz? İlk cevap, karakterlerin ASCII tablosundaki sayısal karşılıkları oldu. Hoca bunu sormadığını, işaretsiz bir tanımlamada saklanabilecek sayı aralığını sorduğunu belirtti. İkinci cevap 127'ye kadar oldu, bu da yanlıştı. **Cevap:** 0 ile 255 arası. Yalnızca `char` tanımlarsanız aralık -128 ile 127 olur.

1 byte 8 bit olduğundan (ara adım eklendi) $2^8 = 256$ farklı değer saklanır. İşaretsiz tanımlamada bunlar 0'dan $2^8 - 1 = 255$'e kadar gider. İşaretli tanımlamada aralık $-2^7 = -128$ ile $2^7 - 1 = 127$ olur.

> **Ders sonrası not:** Hoca yalın `char`'ın -128 ile 127 aralığını sakladığını söyledi. Doğrusu büyük olasılıkla şudur: Yalın `char`'ın işaretli mi işaretsiz mi olduğu derleyiciye ve mimariye bağlıdır. Çoğu masaüstü ortamda işaretlidir. Aralığın kesin olması gerekiyorsa `signed char` ya da `unsigned char` yazılır.

Soru: `char` tipinin negatif aralığını ne için kullanıyoruz? **Cevap:** Bu tamamen sayısal bir kullanımdır. Karakter değişkenini ASCII ile ilgili bir karakter olarak değil de bir sayı aralığı olarak kullanmak ve bellekte daha az yer ayırmak isterseniz `char` tanımlarsınız. İşlemleriniz -128 ile 127 aralığında kalıyorsa bu yeterlidir. Bu kullanımın ASCII tablosuyla ilgisi yoktur.

Yani `char` normalde karakter tutmak için kullanılan bir tiptir, ama daha az yer kaplaması için sayısal işlerde de kullanılabilir. `int` genel olarak 4 byte, `short int` 2 byte yer kaplar. Bunlar mimariye göre değişebilir. Değişmeyen tek şey `char`'ın 1 byte yer kaplamasıdır. Düşük bellek

kullanımı için bu önemlidir. Karakterlerle ilgili bir iş yapacaksanız ise mutlaka karakter tipinde tanımlamalısınız.

7. Format Belirteçleri

Ekrana formatlı yazdırmak için şu belirteçler kullanılır:

- `%d`: Onluk (decimal) tam sayı.
- `%c`: Karakter.
- `%f`: `float`.
- `%lf`: `double`.
- `%s`: Karakter dizisi, yani string (array of characters).

8. Geliştirme Ortamları

Aradan sonra hoca, C ile program yazmak için hangi ortamların kullanılacağını sordu. Öğrencilerden Dev-C++ ve JetBrains'in CLion'u geldi. CLion'dan derleyici olarak söz edilince hoca önce kavramları ayırdı.

8.1 Derleyici, Editör ve IDE

Derleyici ile geliştirme ortamı (development environment) farklı kavramlardır. İkisinin birlikte kullanıldığı yapılara tümleşik geliştirme ortamı (Integrated Development Environment, IDE) denir. Bir IDE'de üç şey bir arada bulunur:

- **Editör:** Kodu yazdığınız ortamdır. Kodu renklendirebilir, kullanacağınız fonksiyon ya da metotları otomatik olarak önünüze çıkarabilir. Bu otomatik öneriler daha çok nesneye yönelik programlamada kullanılır, çünkü C'de komutlar zaten sınırlıdır.
- **Derleyici:** Kodu derler ve çalıştırma imkânı verir.
- **Hata ayıklayıcı (debugger):** Koddaki hatayı ayıklama işine debug denir. Debugger bu işin aracıdır, kodunuzda hata olup olmadığını tespit etmenizi sağlar.

Derste geçen ortamlar şöyle ayrıldı:

- **IDE:** Dev-C++, CLion, Code::Blocks, CodeLite. Mac tarafında Xcode. VS Code da öğrencilerden öneri olarak geldi.
- **Yalnız editör:** Notepad++. Kullanılabilir, ama IDE değildir.
- **Yalnız derleyici:** GCC. IDE değildir.

Hangisini seçeceğiniz kendi ihtiyacınıza bağlıdır. Önemli olan, içinde bir editör, bir derleyici ve bir hata ayıklayıcı bulunmasıdır.

> **Ek bilgi (derste anlatılmadı):** VS Code büyük olasılıkla tek başına derleyici içermez, temelde bir editördür. C için kullanılacaksa sisteme ayrıca bir derleyici kurmak ve eklentilerle bağlamak gerekir.

8.2 ANSI Standartları

Hoca Microsoft tarafını pek önermediğini, çünkü orada ANSI standartlarının gözetilmediğini söyledi. ANSI standartları, American National Standards Institute tarafından belirlenmiştir.

> **Hocanın vurgusu:** ANSI standartları bizim açımızdan önemlidir. Bu standartlara uygun yazarsanız kodunuz bütün ortamlarda sorunsuz çalışır. Bir işletim sistemine ya da işlemci mimarisine özgü geliştirme yaparsanız diğer ortamlarda sıkıntı yaşarsınız.
>
> **Ders sonrası not:** Hoca standart koda uygun yazılan kodun bütün ortamlarda sorunsuz çalıştığını söyledi. Doğrusu büyük olasılıkla şudur: Standart kaynak kod her ortamda derlenebilir, ama bölüm 2.3'te anlatıldığı gibi her mimari için yeniden derlenmesi gerekir. Taşınabilir olan çalıştırılabilir dosya değil, kaynak koddur. Tiplerin boyutu da bölüm 6.1'de geçtiği gibi mimariye göre değişebilir.

8.3 Terminal ve Online Ortamlar

IDE kullanmak zorunlu değildir. Kodu bir editörde yazıp, sisteminizde derleyici yüklüyse terminalden derleyip çalıştırabilirsiniz. Hoca bunların yeri geldiğinde görüleceğini söyledi. Online ortamlar da vardır.

Hoca, şimdiye kadar çözülen 12-13 kadar, belki daha fazla akış diyagramı probleminin koda dönüştürülüp çalıştırılmasının güzel bir antrenman olacağını söyledi.

9. Koşullu Dallanma: if ve else

Akış kontrolünün (control flow) ilk yapısı koşullu dallanmadır (conditional branching), yani `if` kullanımıdır. Teorik gösterimi şöyledir:

```
if (x)
    statement1;   /* true kolu */
else
    statement2;   /* false kolu */
```

Parantezin içine ifadenizi (expression) yazarsınız, karşılaştırmanız varsa onu yazarsınız. `statement` gösterimleri temsilîdir, yapılacak işlemlere karşılık gelir.

9.1 Hangi Kol Çalışır?

Soru: x'in değeri ne olursa true kolu, ne olursa diğer kol çalışır? **Cevap:** Bir öğrenci doğru cevabı verdi: Sıfır dışında bir değer olursa true kolu, sıfır olursa false kolu çalışır.

> **Hocanın vurgusu:** Bizim için önemli olan bilgi şudur: Parantezin içinden sıfır dışında bir değer gelirse true kolu, sıfır gelirse `else` kolu çalışır. Aynı prensibi `while`'da da göreceksiniz.

Hoca bunu iki örnekle gösterdi:

```
if (-1)
    a = b;   /* bu kol calisir: -1 sifirdan farkli */
else
    b = a;

if (0)
    a = b;
else
    b = a;   /* bu kol calisir: deger sifir */
```

Soru: `if` içine sayı dışında bir şey girersek ne olur, örneğin bir karakter? **Cevap:** Değişkenin sayısal karşılığı neyse ona göre çalışır. Hoca bunu `a` değişkeninin 0 olduğu bir durumla gösterdi ve `if (a)` yazıldığında `a = a - b` satırının çalışacağını söyledi.

9.2 Karşılaştırma İfadesinin Değeri

Karşılaştırma ifadeleri de bir değer döndürür. Hoca bir öğrencinin sorusu üzerine şu örneği yazdı:

```
if (x == 4)
    x = x / 2;
else
    x = x * 2;
```

x'e önce 4 verildiğini varsayalım. Adım adım:

1. `x == 4` karşılaştırması doğrudur, bu yüzden 1 değeri döner.
2. Parantezin içindeki değer sıfırdan farklı olduğu için true kolu çalışır.
3. `x = x / 2` çalışır, yani $x = 4/2 = 2$ olur (ara adım eklendi).

Karşılaştırma yanlış olsaydı 0 değeri dönecek ve `else` koluna gidilecekti.

9.3 Süslü Parantez

Süslü parantez kullanmazsanız koşula yalnızca hemen sonraki tek ifade bağlıdır:

```
if (x)
    statement1;
else
    statement2;
statement3; /* Always executed */
```

Burada `statement3` artık `else` koluna ait değildir, `if`'ten bağımsız olarak her zaman çalışır. `else` koluna dahil etmek istiyorsanız süslü parantez mutlaka gerekir:

```
if (x) {
    statement1;
} else {
    statement2;
    statement3;
}
```

Tek bir iş yapıyorsanız süslü parantezsiz de yazabilirsiniz, bunun bir sakıncası yoktur.

> **Hocanın vurgusu:** Yapacağınız işleri mutlaka süslü parantezler içinde yazın. Tek iş olsa bile süslü parantez kullanmakta fayda var. Hiçbir sözdizimi hatası oluşturmaz, aksine işi garanti altına alır. Ben her zaman süslü parantez kullanarak yazmanızı tavsiye ediyorum.

10. İç İçe if: Üç Sayının En Küçüğü

Hoca iç içe if (nested if) yazmanın bir sorun oluşturmadığını söyleyip küçük bir fonksiyon yazdı.

```
int min(int a, int b, int c) {
    if (a < b)
        if (a < c)
            return a;
        else
            return c;
    else if (b < c)
        return b;
    else
        return c;
}
```

Soru: Bu fonksiyon ne bulur? **Cevap:** Üç sayının en küçüğünü.

Hoca adımları şöyle izledi:

- `a < b` doğruysa bu kolun içinde `a < c` sorulur.
  - `a < c` de doğruysa `a` hem `b`'den hem `c`'den küçüktür, `a` döndürülür.
  - `a < c` yanlışsa en küçük sayı otomatik olarak `c` olur, `c` döndürülür.
- `a < b` yanlışsa `b < c` sorulur.
  - `b < c` doğruysa `b` döndürülür.
  - `b < c` yanlışsa `c` döndürülür.

Bu kodda her `else` bir `if` ile eşleşir. İlk `else`, `a < c` sorusuna; ikinci `else`, `a < b` sorusuna; üçüncü `else` ise `b < c` sorusuna aittir. Hoca iç içe yazıldıkları için bunların biraz karışık göründüğünü söyledi.

> **Ek bilgi (derste anlatılmadı):** Süslü parantez yoksa C'nin eşleştirme kuralı büyük olasılıkla şöyledir: Her `else`, kendisinden önceki en yakın ve henüz eşleşmemiş `if`'e bağlanır. Girintinin bu eşleşmeye etkisi yoktur, girinti yalnız okuyan kişi içindir.

Soru: Burada `main` mi yazıyor? **Cevap:** Hayır, `min` yazıyor, yani minimum. Bu, üç sayıdan en küçüğünü bulan küçük bir fonksiyondur.

10.1 Fonksiyon ve return

Bu fonksiyon ana programdan ayrı yazılmıştır ve ana programın içinde çağrılarak kullanılabilir. Hoca bunu ileriye dönük bir bilgi olarak gösterdi. Bu dönem bütün işler fonksiyon yazmadan, yalnızca ana fonksiyon kullanılarak yapılacak. Modüler programlama konusuna gelindiğinde görüleceği gibi, karmaşıklık yönetilebilsin ve problem kolayca çözülebilsin diye problemler alt problemlere ayrılarak çözülür.

`return`, bir fonksiyondan değer döndürmek için kullanılan anahtar kelimedir. `return a;` derseniz `min` fonksiyonundan `a` değeri ana fonksiyona geri döndürülür.

Aynı işi ana programda yapmak için buradaki işlemleri `main` içine yazmanız yeterlidir. Bu durumda `return` demek yerine değeri ekrana yazdırırsınız:

```
printf("En kucuk = %d", a);
```

Soru: `return` deyince değeri mi verir? **Cevap:** Evet, `return` fonksiyondan bir değer döndürür.

11. Döngüler: while ve do...while

11.1 while

`while` anahtar kelimesinden sonra parantez içinde koşul yazılır, süslü parantez içinde de yapılacak işler (statement) yer alır:

```
while (x < y) {
    x++;
    y--;
}
```

`x`, `y`'den küçük olduğu sürece parantezin içindeki işler yapılır.

11.2 do...while

Soru: `while` ile `do...while` arasındaki fark nedir? **Cevap:** `do...while` önce işi yapar, sonra kontrol eder. Yani yapılacak işi mutlaka bir kez yapar, ondan sonra koşula bakar. `while` ise önce koşulu kontrol eder, koşul sağlanırsa işi yapar.

Hoca farkı görmek için aynı işlemleri `do...while` ile yazdı:

```
do {
    x++;
    y--;
} while (x < y);
```

Burada önce `x` artırılıp `y` azaltılır, sonra kontrol yapılır. Yukarıdaki `while`'da ise önce kontrol yapılır, sonra `x` artırılıp `y` azaltılır.

11.3 Noktalı Virgül

Hoca burada söylemeyi unuttuğu bir kuralı ekledi: Bütün ifadelerden (statement) sonra noktalı virgül kullanılır. `a = b;` ya da `a = 3 + y;` gibi her işlemin sonunda mutlaka noktalı virgül bulunur.

`while` satırının sonuna noktalı virgül konmaz. `do...while`'da ise `while`'ın sonuna noktalı virgül konur. Bu da istisnai durumlardan biridir.

12. Döngüler: for

`for` döngüsünün üç ifadesi vardır: `expression1`, `expression2`, `expression3`. Hoca bunu bir dizinin elemanlarını toplayan örnekle gösterdi:

```
for (i = 0; i < n; i++) {
    toplam = toplam + dizi[i];
}
```

12.1 Çalışma Sırası

Hoca çalışma sırasını soru sorarak kurdu.

Soru: İlk hangisi çalışır? **Cevap:** `expression1`. İlk değer atama (ilklendirme) yapılır. Birden fazla ilklendirme yapılabilir, araya virgül koymak yeterlidir.

Soru: İkinci hangisi çalışır? **Cevap:** `expression2`, yani `i < n` kontrolü. İlklendirme yapıldıktan sonra döngüye girip girmeyeceğimize buradan bakılır.

Soru: Üçüncü sırada ne olur? Gelen cevap "`i` bir artırılır" oldu. **Cevap:** Hayır. Üçüncü sırada gövde (statement) çalışır. Gövde çalıştıktan sonra `expression3` çalışır ve `i` bir artırılır.

Hoca sırayı tekrar etti:

1. `expression1` çalışır.
2. `expression2` kontrol edilir.
3. Gövde çalışır.
4. `expression3` çalışır, `i` bir artırılır.

12.2 i– Yazılırsa

Soru: `i++` yerine `i--` yazsaydınız ne olurdu? **Cevap:** `i` 0'dan başlıyorsa ve `n` pozitif bir sayıysa, `i`'yi her seferinde bir azaltmak koşulu hiç bozmaz. Sonsuz döngü oluşur ve program tamamlanmaz.

> **Ders sonrası not:** Hoca bu durumda sonsuz döngü oluşacağını söyledi. Kavramsal olarak doğrudur, koşul mantıken hiç bozulmaz. Doğrusu büyük olasılıkla şudur: `i` negatife geçtiğinde `dizi[i]` dizinin sınırları dışına erişir, bu yüzden bu örnekteki program gerçekte hatayla sonlanabilir. `int` tipinin sınırı aşıldığında davranış da tanımsızdır.

13. C'de Dizi İndisleri

Soru: `n` terimli bir dizide `i`'yi 0'dan başlatınca sıfırıncı terim gibi bir şey olmuyor mu? Normalde birden başlatmıyor muyduk? **Cevap:** Akış diyagramlarını çizerken diziyi 1'den başlatıp n'e kadar devam ediyordunuz. C programlama dilinde ise ilk eleman sıfırıncı gözde, ikinci eleman birinci gözde durur. Dolayısıyla n'inci eleman $n-1$'inci gözde, en sonda yer alır.

Akış diyagramlarında 1'den n'e kadar kullanmak hâlâ geçerlidir. Kod yazarken ise 1'den n'e yaptığınız uygulamayı 0'dan $n-1$'e revize etmeniz gerekir. Akış diyagramında 0'dan başlatıp $n-1$'e kadar gitmek de yanlış değildir.

14. Derleyici ile IDE Farkı Üzerine Son Soru

Soru: Derleyici ile geliştirme ortamı arasındaki bağlantıyı tam anlayamadım. **Cevap:** Derleyici, yazdığınız kaynak kodu dilin kurallarına göre alır ve makine diline çevirir. Bu işlem bittikten sonra üretilen yeni dosyayı çalıştırırsınız. Program doğru yazıldıysa çalışır, bilgileri alır ve işlemi gerçekleştirir. IDE ise derleyicinin de içinde bulunduğu, kodu yazmanızı sağlayan editörü ve hata ayıklamanızı sağlayan debugger'ı da barındıran komple bir platformdur. Dev-C++, Code::Blocks ve CodeLite bu tür ortamlardır. Bunlar hem kod yazmanızı hem derlemenizi hem de çalıştırmanızı sağlar.

Dersin Toparlaması

Bu derste algoritma tasarımından C diline geçildi. Önce programlama dillerinin biçimsel yapısı, seviyeleri ve derlenen ya da yorumlanan diller olarak ayrımı görüldü. C'nin neden güçlü olduğu ve neden dikkat gerektirdiği açıklandı. Ardından bir C programının iskeleti, `scanf` ve `printf`, operatörler, değişken tipleri ve format belirteçleri verildi. Derleyici, editör ve IDE ayrıldı. Son bölümde `if-else`, iç içe `if`, `while`, `do...while` ve `for` yapıları ile C'de dizi indislerinin 0'dan başlaması işlendi.

Hoca kurallardan yalnızca yazımla ilgili olanları hızlıca anlattığını, kod yazabilecek noktaya gelindiğini düşündüğünü söyledi. C'ye dair başka özellikler de ilerleyen derslerde anlatılacak

ve uygulama derslerinde birlikte kod yazılacak. Bu aşamada algoritma tasarlamak kod yazmaktan daha önemlidir. Yine de çözülen akış diyagramlarını bugün öğrenilen bilgilerle yavaş yavaş koda dönüştürmek gerekir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.